🔥 **STAGE 2 — ASYNC SCRAPING (MASTER SUMMARY & SYSTEM BLUEPRINT)**
*(This is NOT notes. This is a **mental operating system** you can reuse forever.)*

You should be able to **rebuild any async scraper system from zero** after reading this.

🧠 STAGE 2 — SYSTEM GOAL (LOCK THIS FIRST)

> **Build a scraper that is fast, safe, observable, and scalable — without getting banned or crashing.**

Not:

- fastest possible ❌
- clever syntax ❌

But:

- **predictable**
- **controllable**
- **defensible to a client**

🏗️ OVERALL ARCHITECTURE (BIG PICTURE)

```
URLs
  ↓
main()  ── system orchestrator
  ├── concurrency control (Semaphore)
  ├── rate limiting (async sleep)
  ├── ClientSession (single)
  ├── task creation
  ├── asyncio.gather()
  ├── performance measurement
  └── final reporting
        ↓
fetch_html() ── network worker (async)
  ├── async request
  ├── timeout handling
  ├── status handling (200 / 403 / 429)
  ├── retry + backoff
  ├── empty HTML detection
  ├── logging (start / end / fail)
  └── returns: HTML or None
        ↓
parse_html() ── CPU work (sync)
  ├── BeautifulSoup
  └── DOM creation
        ↓
extract_data() ── business logic (sync)
  └── structured data
```

Golden rule:

> `main()` controls the system
> `fetch_html()` faces the internet
> parsing stays **sync**

1️⃣ ASYNC MENTAL MODEL (FOUNDATION)

🔑 What problem async solves

- Network waits waste time
- CPU sits idle
- Async **overlaps waiting**

🧠 Real-life analogy

- Sync = calling and waiting ☎️
- Async = WhatsApp messages 💬

🧱 Concepts

- **Blocking I/O**: network, disk, DB
- **Event loop**: traffic controller

🔒 Rule

> **Async is for waiting, not for thinking.**

2️⃣ ASYNC SYNTAX (NO CONFUSION ZONE)

🔤 Core syntax

```
async def func():        # function can pause
    await something()    # pause point
```

```
asyncio.run(main())     # start event loop
```

```
await asyncio.gather(*tasks, return_exceptions=True)
```

❌ Never do

- Call async function without `await`
- Use `time.sleep()` in async
- Mix sync mindset with async flow

3️⃣ AIOHTTP BASICS (NETWORK LAYER)

🔑 Why `ClientSession`

- TCP connection reuse
- Faster
- Server-friendly

```
async with aiohttp.ClientSession(timeout=timeout) as session:
```

🔑 Why `async with session.get()`

- Ensures connection closes
- Prevents socket leaks

```
async with session.get(url) as response:
    html = await response.text()
```

🌐 Status handling

- `200` → OK (but still validate HTML)

- `403` → blocked (do not retry)
- `429` → too fast (slow down)
- timeout → temporary failure

4️⃣ CONCURRENCY CONTROL (CAPACITY)

🔑 Semaphore meaning

> “Itne hi kaam ek saath.”

```
sem = asyncio.Semaphore(5)
```

✅ Correct placement

```
async with sem:
    async with session.get(url):
```

❌ Wrong placement

- Outside fetch
- After response

🧠 Rule

> Semaphore controls **how many at once**, not speed over time.

5️⃣ FAILURE HANDLING (PIPELINE SAFETY)

🔑 Philosophy

> **Failure is data, not death.**

🧩 Partial failures

```
results = await asyncio.gather(*tasks, return_exceptions=True)
```

- Length preserved
- One failure ≠ crash

🧩 Timeout handling

```
timeout = aiohttp.ClientTimeout(total=10)
```

```
except asyncio.TimeoutError:
    return None
```

🧩 Empty HTML detection (CRITICAL)

```
if not html or len(html.strip()) < 100:
    return None
```

> 200 OK ≠ usable data

🧩 Status classification

- `403` → skip
- `429` → slow down
- `5xx` → retry possible

6️⃣ ASYNC + PARSING BOUNDARY (DISCIPLINE)

🔑 Core rule

```
Network = async
Parsing = sync
```

❌ Never do

```
async def parse_html(html):
```

✅ Always do

```
def parse_html(html):
```

🧠 Why

- Parsing is CPU-bound
- Async gives zero benefit
- Adds complexity only

7️⃣ RATE LIMIT RESPECT (SURVIVAL)

🔑 Two controls (both required)

Tool	Controls
Semaphore	concurrency
asyncio.sleep	speed over time

🧩 Async sleep

```
await asyncio.sleep(1)
```

❌ `time.sleep()` blocks event loop
✅ `asyncio.sleep()` is non-blocking

🧩 Adaptive slowdown

```
if response.status == 429:
    await asyncio.sleep(5)
```

🧩 Retry strategy

- Retry only temporary failures
- Max retries = 2–3
- Backoff delay increases

```
try → fail → wait → retry → wait longer → stop
```

8️⃣ LOGGING IN ASYNC WORLD (OBSERVABILITY)

🔑 Why print fails

- No order
- No timing
- No identity

🧩 Logging basics

```
import logging

logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s | %(levelname)s | %(message)s"
)
```

🧩 What to log (ONLY)

1. START (url)
2. END (url + duration)
3. FAILURE (reason)
4. STATUS (success / fail)

🧩 Timing + correlation

```
start = time.perf_counter()
# work
end = time.perf_counter()
duration = end - start
```

Every log must include:

- URL
- duration
- outcome

9️⃣ PERFORMANCE PROOF (MANDATORY)

🔑 Core truth

> **Async is faster because it overlaps waiting.**

🧩 Measuring time

```
start = time.perf_counter()
await run_async()
end = time.perf_counter()
```

Why `perf_counter` ?

- Monotonic
- High precision
- Benchmark-safe

🧩 Fair benchmarking rules

- Same URLs
- Same headers
- Same retries
- Same machine
- Only execution model differs

🧩 Presenting proof (client)

```
Sync time: 18s
Async time: 6s
Speedup: ~3x
Reason: network waiting overlapped
```

❌ No jargon
❌ No raw logs

🔟 FREELANCE READINESS (SYSTEM THINKING)

🔑 First question to client

> “Kitni der me chahiye?”

🧠 Estimation logic

```
total_urls / safe_concurrency = time
```

Always add buffer for:

- sleep
- retries
- failures

🧩 Professional response structure

1. Plan
2. Speed estimate
3. Safety
4. Failure handling

🧩 What you promise

- Clean data
- Failure report
- Honest estimate

Never promise:

- No bans
- Fixed speed
- Absolute guarantees

🧱 CANONICAL FETCH FUNCTION (FINAL FORM)

```python
async def fetch_html(session, url, sem, retries=2):
    for attempt in range(retries + 1):
        start = time.perf_counter()
        try:
            async with sem:
                async with session.get(url) as response:

                    if response.status == 403:
                        return None

                    if response.status == 429:
                        await asyncio.sleep(5)
                        continue

                    html = await response.text()

                    if not html or len(html.strip()) < 100:
                        return None

                    return html

        except asyncio.TimeoutError:
            await asyncio.sleep(2)

        except aiohttp.ClientError:
            return None

        finally:
            end = time.perf_counter()
```